

STUDY PLANNER

StudyFlow

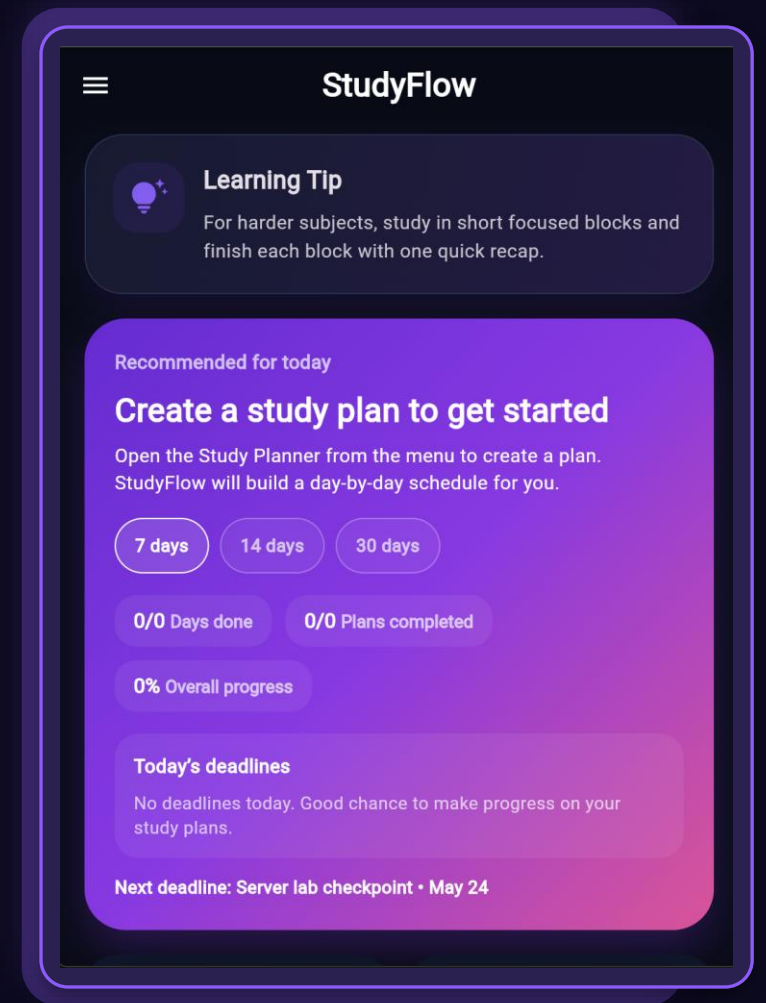
A mobile study planner built with Flutter

Phan Ngoc Phuoc Loc
Hao Chen

Mobile Applications · TAMK / THWS

Examiners: Peter Braun & Tony Torp

30 May 2026



What we'll cover

1

Domain & goal

What StudyFlow does and who it's for

3

Live demo

The app running on the emulator

5

Data layer

SQLite seeded from a JSON asset

2

Architecture

Layered structure & separation of concerns

4

State management

Provider + persistence

6

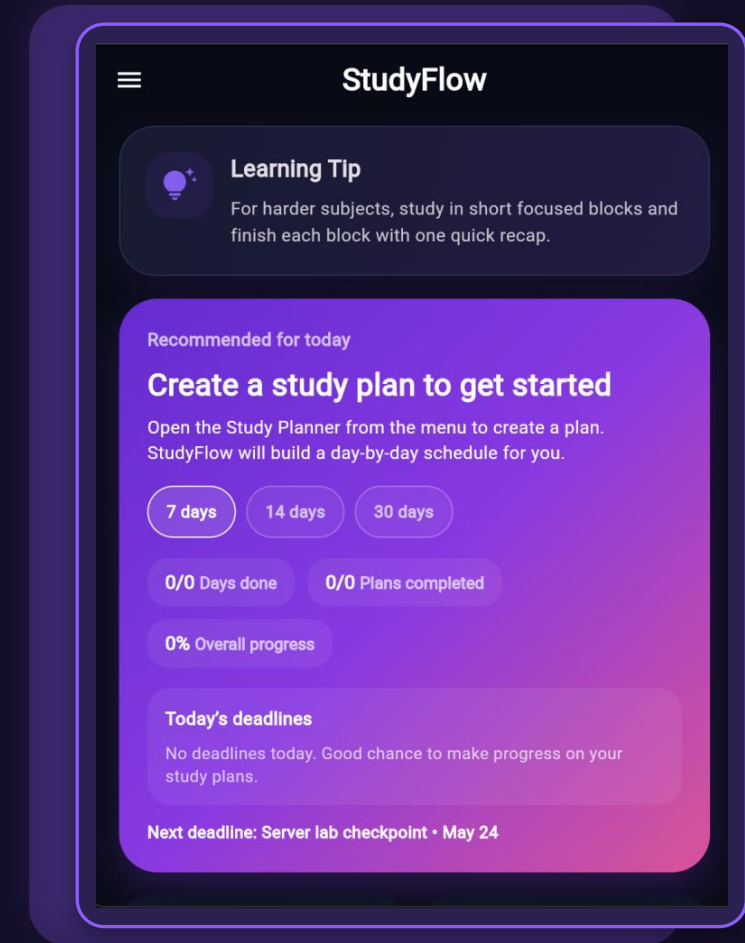
Testing

Model tests, providers tests, validators tests, service tests, widget tests

Plan your studies, day by day

Domain — productivity & learning

- Create study plans from templates and seed data
- Track daily progress, deadlines and completion
- Browse a paginated library of learning tips
- Mark favourites, switch light / dark theme
- Settings, About & feedback screens



Built with

Flutter

Cross-platform UI toolkit; runs on Android & iOS

provider

App-wide state management (course requirement)

go_router

Declarative named routes & deep links

sqflite (SQLite)

Local database, seeded from a JSON asset

shared_preferences

Persist theme, notifications & favourites

url_launcher

Open mail client & GitHub from the app

ARCHITECTURE

Layered architecture, clean separation

```
lib/  
  
  models/  
    study_plan.dart · plan_item.dart ...  
  
  services/  
    database_service.dart (SQLite)  
    mock_data_service.dart · settings...  
  
  providers/  
    study_flow_controller.dart  
    app_settings_provider.dart ...  
  
  validators/  
    study_plan_validator.dart  
  
  screens/ · widgets/  
  
  router/ · theme/
```

DATA LAYER models/ · services/
Model classes, JSON parsing, SQLite & persistence

LOGIC LAYER providers/
ChangeNotifier controllers hold & update state

UI LAYER screens/ · widgets/
Screens and reusable widgets render state

Supporting: router/ (go_router) · theme/ (light & dark) · validators/ (logic outside widgets)

REQUIREMENTS

Every technical requirement, covered



Drawer on first screen `AppDrawer` on the dashboard



Scrollable list view `Dashboard ListView`



Data from DB + JSON `SQLite` seeded from assets `JSON`



Images (URL + assets) `Image.network` with icon fallback



Custom list-tile widgets `StudyPlanCard`, `StudyTipListTile`



Detail screen `PlanDetailPage` `(/plan/:id)`



Validated input (external) `study_plan_validator.dart`



Light / dark theme `AppSettingsProvider` + themes



State management `provider` (`ChangeNotifier`)



Named routes `go_router` route table



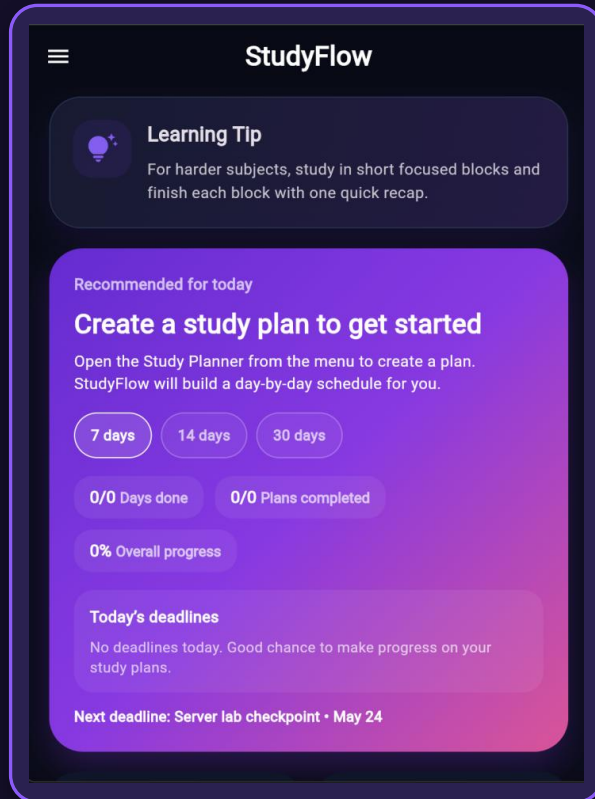
Shared preferences Theme, notifications, favourites



Tests passing `flutter test` – 29 / 29

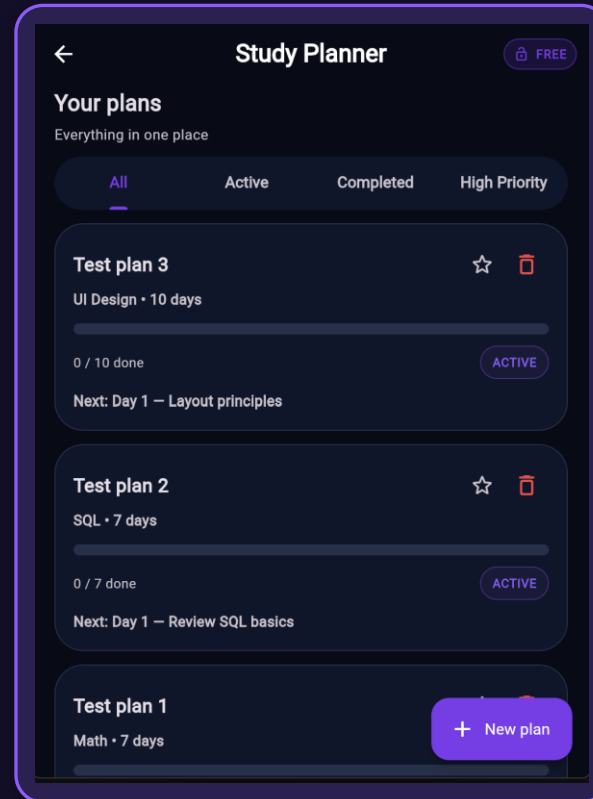
WALKTHROUGH

A look at the app



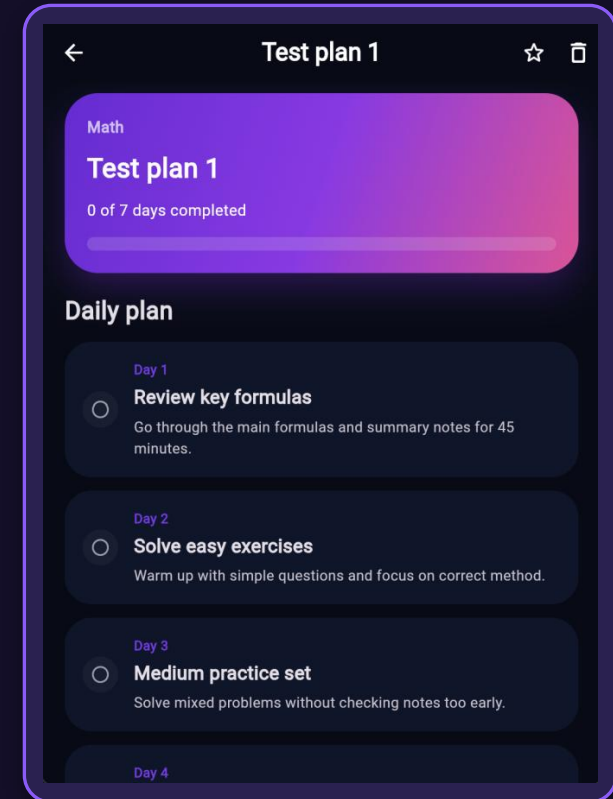
Dashboard

Drawer, scrollable list, today's recommendation



Study Planner

Custom plan cards, tab filters, create plan



Plan Detail

Second screen — day-by-day breakdown

Provider drives the whole app

- **MultiProvider at the app root**

Settings, study plans, favourites & premium

- **ChangeNotifier controllers hold logic**

notifyListeners() rebuilds only what's watching

- **Screens read with context.watch<T>()**

e.g. theme switch repaints MaterialApp instantly

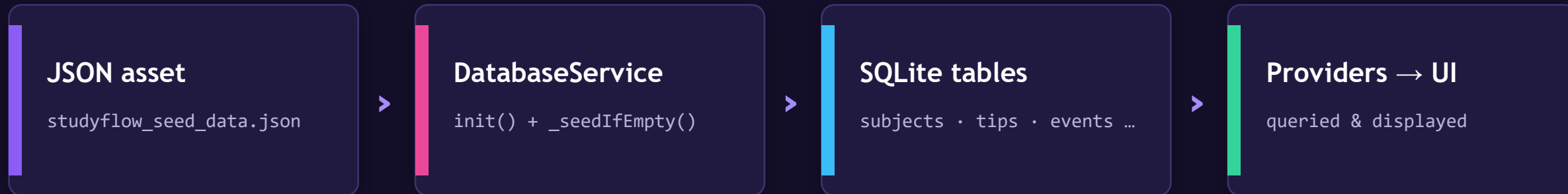
- **State persists across launches**

Loaded once in main() before runApp()

```
runApp(  
  MultiProvider(  
    providers: [  
      ChangeNotifierProvider<AppSettingsProvider>.value(value: settings),  
      ChangeNotifierProvider<StudyFlowController>.value(value: studyFlow),  
      ChangeNotifierProvider<FavouritesProvider>.value(value: favourites),  
      ChangeNotifierProvider(create: (_) => PremiumController()),  
    ],  
    child: StudyFlowRoot(router: router),  
  ), // MultiProvider  
);
```

```
lib/main.dart  
  
final settings = AppSettingsProvider();  
  
await settings.load();  
  
final studyFlow = StudyFlowController();  
  
await studyFlow.initialize();  
  
runApp(  
  MultiProvider(  
    providers: [  
      ChangeNotifierProvider  
        .value(value: settings),  
      ChangeNotifierProvider  
        .value(value: studyFlow),  
    ],  
    child: StudyFlowRoot(router: router),  
  ),  
);
```

SQLite database, seeded from JSON



Why this matters

Satisfies “data comes from a database populated with a JSON file in the assets.” On first launch `DatabaseService` creates four tables and seeds them in a single batch; later launches skip seeding via `_seedIfEmpty()`.

Loading, error & empty states



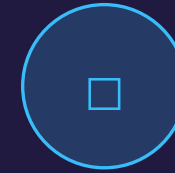
Loading

Async work shows a progress indicator while data loads from the database and feed.



Error

Failures show a clear message with a retry option instead of a blank or broken screen.



Empty

No plans yet? A friendly empty state guides the user to create their first plan.

Tested across every layer

All test passed

```
$ flutter test
```

✓ All tests passed!

What's covered

- ✓ **Model** StudyPlan toJson / fromJson round-trips
- ✓ **Validators** All validator functions (topic, feedback)
- ✓ **Service** MockDataService data access
- ✓ **Providers** Settings, favourites, study-flow controller
- ✓ **Widget tests** Create-plan form, settings, planner, tips, help

Thank you

Questions & source-code walkthrough

Phan Ngoc Phuoc Loc
Hao Chen

github.com/Lockz178/studyflow-app

